

# ANU College of Engineering and Technology

ACHARYA NAGARJUNA UNIVERSITY  
Department of Computer Science & Engineering  
Course file for Data Structures & Algorithms Lab

Course : II/IV B.Tech I semester  
Branch : CSE R20 Regulation  
Subject name : Data Structures & Algorithms Lab  
Subject code : CSE251 Lab (R20)  
Prepared by : Dr K.CHAITANYA  
Assistant Professor  
Department of CSE  
ANU College of Engineering & Technology  
Acharya Nagarjuna University.

### **Vision of the University:**

To generate sources of knowledge that dispel ignorance and establish truth through teaching, learning and research

### **Mission of the University:**

To promote a bank of human talent in diversified faculties – Commerce & Management Studies, Education, Engineering & Technology, Humanities, Law, Natural Sciences, Pharmacy, Physical Education & Sports Sciences, Physical Sciences and Social Sciences that would become an investment for a prosperous society.

### **Vision of the College:**

ANU College of Engineering & Technology is started with an aim of imparting technical values in the students, who can change the shape of global scenario in engineering arena.

### **Mission of the College:**

- To educate students for careers of leadership, innovation in engineering and its related fields.
- To expand the base of engineering knowledge through original research and by developing technology to serve the needs of society.

### **Vision of the Department:**

To be in the frontiers of Computer Science and Engineering with academic excellence and Research.

### **Mission of the Department:**

The mission of the Computer Science and Engineering Department is to:

- Educate students with the best practices of Computer Science by integrating the latest research into the curriculum.
- Develop professionals with sound knowledge in theory and practice of Computer Science and Engineering.
- Facilitate the development of academia-industry collaboration and societal outreach programs.
- Prepare students for full and ethical participation in a diverse society and encourage lifelong learning.

### **3. PROGRAM EDUCATIONAL OBJECTIVES (PEO's):**

- PEO1: Practice engineering in a broad range of industrial, societal and real world applications.
- PEO2: Pursue advanced education, research and development, and other creative and innovative efforts in science, engineering, and technology, as well as other professional careers.
- PEO3: Conduct themselves in a responsible, professional, and ethical manner.
- PEO4: Participate as leaders in their fields of expertise and in activities that support service and economic development throughout the world.

### **PROGRAM OUTCOMES (PO's):**

|            |                                                                                                                                                                                  |
|------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>PO1</b> | Engineering knowledge: Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems. |
|------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

|             |                                                                                                                                                                                                                                                                                                   |
|-------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>PO2</b>  | Problem analysis: Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.                                                                 |
| <b>PO3</b>  | Design/development of solutions: Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.         |
| <b>PO4</b>  | Conduct investigations of complex problems: Use research-based knowledge and Research methods including design of experiments, analysis and interpretation of data and synthesis of the information to provide valid conclusions.                                                                 |
| <b>PO5</b>  | Modern tool usage: Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.                                                                 |
| <b>PO6</b>  | The engineer and society: Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.                                                               |
| <b>PO7</b>  | Environment and sustainability: Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.                                                                                   |
| <b>PO8</b>  | Ethics: Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.                                                                                                                                                                    |
| <b>PO9</b>  | Individual and team work: Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.                                                                                                                                                   |
| <b>PO10</b> | Communication: Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions. |
| <b>PO11</b> | Project management and finance: Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.                                               |
| <b>PO12</b> | Life-long learning: Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.                                                                                                                 |

### **PROGRAM SPECIFIC OUTCOMES (PSO's):**

|             |                                                                                                   |
|-------------|---------------------------------------------------------------------------------------------------|
| <b>PSO1</b> | An ability to Understand the theoretical and mathematical concepts to analyze real time problems. |
| <b>PSO2</b> | An Ability to Design and Analyze systems based on the theoretical and Practical Knowledge         |

## **Syllabus:**

|         |                                  |
|---------|----------------------------------|
| CSE 251 | Data structures & Algorithms Lab |
|---------|----------------------------------|

### **COURSE OBJECTIVES:**

- To impart the basic concepts of data structures and algorithms
- To understand concepts about searching and sorting techniques
- To Understand basic concepts about stacks, queues, lists, trees and graphs
- To understanding about writing algorithms and step by step approach in solving problems
- With the help of fundamental data structures

### **COURSE OUTCOMES:**

- Understand basic data structures such as arrays, linked lists, stacks and queues.
- Implement the stack, Queue and their applications
- Implement various types of linked lists and their applications
- Ability to have knowledge of tree and graphs concepts.
- Apply Algorithm for solving problems like sorting, searching, insertion and deletion of data

### **LIST OF ACTIVITIES:**

- 1) Write a C program to perform insertion, deletion, display operations on Array Data Structure.
- 2) Write a C program to perform insertion, deletion, display operations on stack Data Structure using arrays.
- 3) Write a C program to convert given infix expression into postfix expression and evaluate the postfix expression.
- 4) Write a C program to convert given infix expression into prefix expression and evaluate the prefix expression.
- 5) Write a C program to perform insertion, deletion, display operations on queue Data Structure using arrays.
- 6) Write a C program to perform insertion, deletion, display operations on circular Queue Data Structure using arrays.
- 7) Write a C program to perform insertion, deletion, display operations on single linked list Data Structure.
- 8) Write a C program to perform insertion, deletion, display operations on circular single linked list Data Structure.

- 9) Write a C program to perform insertion, deletion, display operations on double linked list Data Structure.
- 10) Write a C program to perform insertion, deletion, display operations on circular double linked list
- 11) Implement a polynomial ADT and write a program to read two polynomials and print them, adds the polynomials, prints the sum, multiply the polynomials and print the product.
- 12) Implement the following sorting operations:- Shell Sort, (b) Heap Sort (c) Merge Sort (d) Quick Sort e) Radix Sort
- 13) Implement the following sorting operations:- (a) Sequential search (b) Binary Search
- 14) Write a C program for open addressing.
- 15) Implement Binary Tree ADT and write a program that reads postfix Arithmetic
- 16) Expression form, builds the expression tree and performs tree Traversal on it.
- 17) Implement Binary search Tree ADT and write a program that interactively allows Insertion (b) Deletion (c) Find\_min (d) Find\_max (e) Find operations
- 18) Write a C program for Graph Traversals: a) DFS b) BFS

### CO-PO Mapping:

|     | PO1 | PO2 | PO3 | PO4 | PO5 | PO6 | PO7 | PO8 | PO9 | PO10 | PO11 | PO12 | PSO1 | PSO2 |
|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|------|------|------|------|------|
| CO1 | 3   | 2   | 2   | 1   | 2   |     |     |     |     |      |      |      |      |      |
| CO2 | 3   | 2   | 2   | 1   | 2   |     |     |     |     |      |      |      |      |      |
| CO3 | 3   | 2   | 2   | 1   | 2   |     |     |     |     |      |      |      |      |      |
| CO4 | 3   | 2   | 2   | 1   | 2   |     |     |     |     |      |      |      |      |      |
| CO5 | 2   | 2   | 2   | 1   | 2   |     |     |     |     |      |      |      |      |      |

### Lab Manual with viva questions:

1. **Problem Statement:** 1) Write a C program to perform insertion, deletion, display operations on Array Data Structure.

```
#include <stdio.h>
#include <stdlib.h>
int main()
{
    int a[100];
    int element,i,loc,size,n=0,j=0;
    printf("Enter the size of an array\n");
    scanf("%d",&size);
```

```

printf("Enter %d array elements\n",size);
for(i=0;i<size;i++)
{
    scanf("%d",&a[i]);
}
printf("List before Insertion: ");
for(i=0;i<size;i++)
{
    printf("%d ",a[i]);
}
printf("\nEnter an element to insert\n");
scanf("%d",&element);
printf("Enter a position to insert an element %d\n",element);
scanf("%d",&loc);
loc--;
for(i=size-1;i>=loc;i--)
{
    a[i+1]=a[i];
}
a[loc]=element;
printf("\nList after Insertion: ");
for(i=0;i<size+1;i++)
{
    printf("%d ",a[i]);
}
printf("\nEnter an element to delete\n");
scanf("%d",&n);
i=0;
for(i=0;i<size;i++)
{
    if(a[i]==n)
    {
        for(j=i;j<(size-1);j++)
        {
            a[j]=a[j+1];
        }
        break;
    }
}
printf("List after deletion\n");
for(i=0;i<(size-1);i++)
{
    printf("%d ",a[i]);
}
return 0;
}

```

Output:

Enter the size of an array  
Enter 5 array elements  
List before Insertion: 3 5 7 1 9  
Enter an element to insert  
Enter a position to insert an element 6

List after Insertion: 3 5 6 7 1 9  
Enter an element to delete  
List after deletion  
3 5 6 7

### **Viva Questions:**

**1. Define array?**

**2. How memory is allocated for an array?**

**2.Problem Statement:** Write a menu driven program for stack ADT to perform push, pop and display using arrays

**Requirements:** H/W : Pentium IV.  
S/W : Turbo C++ Compiler  
Library : conio.h, stdio.h, process.h

**Algorithm: push operation**

Step 1:if(top==size)

Write"stack overflow"

Step 2:Read "element"

Increment top by 1 and insert element at top of the stack

**Pop operation**

Step 1:if top==--1

Then write "stack underflow"

Step 2:Decrement top by 1

**Program:** #include<stdio.h>  
#include<conio.h>  
#include<process.h>  
#define size 20  
void push();

```

void pop();
void display();
int a[size],top=-1;
void main()
{
    int ch,c;
    do
    {
        printf("stack operations\n");
        printf("1.push\n");
        printf("2.pop\n");
        printf("3.display\n");
        printf("enter your choice\n");
        scanf("%d",&ch);
        switch(ch)
        {
            case 1:push();
                break;
            case 2:pop();
                break;
            case 3:display();
                break;
            case 4:exit(0);
                break;
            default:printf("enter correct choice\n");
                break;
        }
        printf("do you want to continue y=1 n=0\n");
        scanf("%d",&c);
    }while(c==1);
    getch();
}

void push()
{
    if(top>=size)
        printf("stack overflow\n");
    else
    {
        top++;
        printf("enter the element to insert at top\n");
        scanf("%d",&a[top]);
    }
}

void pop()
{
    if(top== -1)
        printf("stack underflow\n");
    else

```



```

{
printf("the deleted element is %d",a[top]);
top--;
}
}
void display()
{
int i;
printf("the elements in the stack are\n");
for(i=top;i>=0;i--)
printf("%d\n",a[i]);
}

```

### Input & Output:

Stack operations

1.push

2.pop

3.display

Enter your choice

1

Enter the element to insert at top

10

Do you want to continue y=1 n=0

1

Stack operations

1.push

2.pop

3.display

Enter your choice

1

Enter the element to insert at top

20

Do you want to continue y=1 n=0

1

Stack operations

1.push

2.pop

3.display

Enter your choice

1

Enter the element to insert at top

30

Do you want to continue y=1 n=0

1

Stack operations

1.push

2.pop

3.display

Enter your choice  
3  
The elements in the stack are  
30  
20  
10  
Do you want to continue y=1 n=0  
0

**Viva Questions:**

- 1.write the applications of single linked list?
- 2.write the applications of stacks?
- 3.explain recursion ?

**3.Problem Statement:**write a program that reads an infix arithmetic expression of variables,constants and converts it into the corresponding postfix form and evaluate the postfix expression.

**Requirements:** H/W : Pentium IV.  
S/W : Turbo C++ Compiler  
Library : conio.h, stdio.h, process.h

**Algorithm: infix to postfix conversion**

**STEP 1 :** Read the given infix expression into string called infix.

**STEP 2 :** Reverse the infix string and read one character at a time and perform the following operations :

If the read character is an operand, then add the operand to the prefix string.

If the read character is not an operand, then check

If the stack is not empty and precedence of the top of the stack operator is higher than the read operator,

then pop the operator from stack and add this operator to the prefix string.

Else push the operator onto the stack.

**STEP 3 :** Repeat **STEP 2** till all characters are processed from the input string.

**STEP 4 :** If stack is not empty, then pop the operator from stack and add this operator to the prefix string.

**STEP 5 :** Repeat **STEP 4** till all the operators are popped from the stack.

**STEP 6 :** Reverse the prefix string and display the result of the given infix expression or the resultant prefix expression stored in a string called prefix from this algorithm.

**Postfix evaluation**

- Scan the `Postfix string` from left to right.
- Initialise an empty stack.
- If the scanned character is an operand, add it to the `stack`. If the scanned character is an operator, there will be atleast two operands in the `stack`.
  - If the scanned character is an Operator, then we store the top most element of the `stack(topStack)` in a variable `temp`. Pop the stack. Now evaluate `topStack(Operator)temp`. Let the result of this operation be `retVal`. Pop the stack and Push `retVal` into the stack.

Repeat this step till all the characters are scanned.

- After all characters are scanned, we will have only one element in the stack. Return `topStack`.

**Program:**

```
#include<stdio.h>
#include<conio.h>
#include<ctype.h>
void push(int);
void pop();
int symbol(char);
void evaluate(char[]);
void push1(int);
int pop1();
int operation(int,int,char);
char po[20],c[20],s[10];
int j=0,i=0,top=-1,stack[10],t=-1;
void main()
{
printf("enter the infix expression\n");
gets(c);
while(c[i]!='\0')
{
if(isdigit(c[i]))
{
po[j]=c[i];
j++;
i++;
}
else if(c[i]=='(')
push(c[i]);
else if(c[i]==')')
{
while(s[top]!='(')
pop();
pop();
i++;
}
```

```

}
else
{
if(top== -1)
{
top++;
s[top]=c[i];
i++;
}
else
{
while(symbol(s[top])>=symbol(c[i]))
{
pop();
}
push(c[i]);
}
}
}
while(s[top]>-1)
pop();
po[j]='\0';
printf("the postfix expression is \n");
puts(po);
evaluate(po);
getch();
}
int symbol(char op)
{
if(op=='^')return 3;
else if(op=='*' | op=='/')return 2;
else if(op=='+' | op=='-')return 1;
else return 0;
}

void pop()
{
if(s[top]!='(')
{
po[j]=s[top];
top--;
j++;
}
else
top--;
}
void push(int c)
{

```

```

top++;
s[top]=c;
i++;
}

void evaluate(char e[])
{
int i=0,a,b,c,z;
while(e[i]!='\0')
{
if(isdigit(e[i]))
{
z=e[i]-48;
push1(z);
i++;
}
else
{
b=pop1();
a=pop1();
c=operation(a,b,e[i]);
push1(c);
i++;
}
}
printf("postfix evaluation=%d",stack[t]);
}

void push1(int x)
{
t++;
stack[t]=x;
}

int pop1()
{
int z;
z=stack[t];
printf("%d\n",z);
t--;
return(z);
}

int operation(int a,int b,char op)
{
int c;
if(op=='*')
c=a*b;
else if(op=='/')
c=a/b;
else if(op=='+')

```

```

    c=a+b;
    else
    c=a-b;
    printf("c=%d",c);
    return(c);
}

```

#### **Input & Output:**

Enter infix expression

1+2\*3

Postfix expression 123\*+

Postfix evaluation=7

#### **Viva Questions:**

1.define reverse polish notation?

**4.Problem Statement:** write a program that reads an infix arithmetic expression of variables,constants and converts it into the corresponding prefix form and evaluate the prefix expression.

#### **Algorithm:**

##### **Infix to prefix conversion**

Reverse the input string.

2) Examine the next element in the input.

3) If it is operand, add it to output string.

4) If it is Closing parenthesis, push it on stack.

5) If it is an operator, then

i) If stack is empty, push operator on stack.

ii) If the top of stack is closing parenthesis, push operator on stack.

iii) If it has same or higher priority than the top of stack, push operator on stack.

iv) Else pop the operator from the stack and add it to output string, repeat step 5.

6) If it is a opening parenthesis, pop operators from stack and add them to output string until a closing parenthesis is encountered. Pop and discard the closing parenthesis.

7) If there is more input go to step 2

8) If there is no more input, unstack the remaining operators and add them to output string.

9) Reverse the output string.

#### **Prefix evaluation**

- Reverse the prefix notation
- Scan the prefix `string` from left to right.
- Initialise an empty stack.
- If the scanned character is an operand, add it to the `stack`. If the scanned character is an operator, there will be atleast two operands in the `stack`.
  - If the scanned character is an Operator, then we store the top most element of the `stack(topStack)` in a variable `temp`. Pop the stack. Now evaluate `topStack(Operator)temp`. Let the result of this operation be `retVal`. Pop the stack and Push `retVal` into the stack.

Repeat this step till all the characters are scanned.

- After all characters are scanned, we will have only one element in the stack. Return `topStack`.

**Requirements:** H/W : Pentium IV.  
 S/W : Turbo C++ Compiler  
 Library : conio.h, stdio.h, ctype.h

**Program:** /\*infix to prefix conversion(with and without paranthesis)&prefix evaluation\*/

```
#include<stdio.h>
#include<conio.h>
#include<ctype.h>
#include<string.h>
void push(int);
void pop();
int symbol(char);
void evaluate(char[]);
void push1(int);
int pop1();
int operation(int,int,char);
char pre[20],c[20],s[10];
int j=0,i=0,top=-1,stack[10],t=-1;
void main()
{
printf("enter the infix expression\n");
gets(c);
strrev(c);
puts(c);
while(c[i]!='\0')
{
```

```

if(isdigit(c[i]))
{
    pre[j]=c[i];
    j++;
    i++;
}
else if(c[i]=='')
{
    push(c[i]);
    i++;
}
else if(c[i]=='(')
{
    while(s[top]!='')
    pop();
    pop();
    i++;
}
else
{
    if(top== -1)
    {
        push(c[i]);
        i++;
    }
    else
    {
        while(symbol(s[top])>symbol(c[i]))
            pop();
        push(c[i]);
        i++;
    }
}
}
while(s[top]>-1)
pop();
pre[j]='\0';
strrev(pre);
printf("the prefix expression is \n");
puts(pre);
evaluate(pre);
getch();
}
int symbol(char op)
{
    if(op=='^')return 3;
    else if(op=='*' || op=='/')return 2;
    else if(op=='+' || op=='-')return 1;
}

```



```
else return 0;
}
```

```
void pop()
{
    if(s[top]!='')
    {
        pre[j]=s[top];
        top--;
        j++;
    }
    else
        top--;
}
void push(int c)
{
    top++;
    s[top]=c;
}
```

```
void evaluate(char e[])
{
    int i=0,a,b,c,z;
    strrev(e);
    while(e[i]!='\0')
    {
        if(isdigit(e[i]))
        {
            z=e[i]-48;
            push1(z);
            i++;
        }
        else
        {
            a=pop1();
            b=pop1();
            c=operation(a,b,e[i]);
            push1(c);
            i++;
        }
    }
    printf("prefix evaluation=%d",stack[t]);
}
void push1(int x)
{
    t++;
    stack[t]=x;
}
```

```

int pop1()
{
int z;
z=stack[t];
t--;
return(z);
}
int operation(int a,int b,char op)
{
int c;
if(op=='*')
c=a*b;
else if(op=='/')
c=a/b;
else if(op=='+')
c=a+b;
else
c=a-b;
return(c);
}

```

#### **Input & Output:**

enter the infix expression  
 1+2/3  
 3/2+1  
 The prefix expression is  
 +1/23  
 Prefix evaluation =1

**Viva Questions:**1.define prefix notation?

**5.Problem Statement:** implement queue ADT using Arrays.

**Requirements:** H/W : Pentium IV.  
 S/W : Turbo C++ Compiler  
 Library : conio.h, stdio.h

**Algorithm:** insert into the queue

Step 1: if rear is equal to size

Then write "queue is full"

Step 2: Increment rear by 1 and insert element at rear position

### **Delete element from the queue**

**Step 1:**if(front== -1)

Then write "queue underflow"

Step 2:increment front by 1

### **Program:**

```
#include<stdio.h>
#include<conio.h>
#define size 5
void insertq();
void deleteq();
void display();
int queue[size],front=-1,rear=-1;
void main()
{
    int ch,c;
    do
    {
        printf("enter the queue operations\n");
        printf("1.insert\n");
        printf("2.delete\n");
        printf("3.display\n");
        printf("enter your choice\n");
        scanf("%d",&ch);
        switch(ch)
        {
            case 1:insertq();
                break;
            case 2:deleteq();
                break;
            case 3:display();
                break;
            default:printf("enter correct choice\n");
        }
        printf("do you want continue y=1 n=0\n");
        scanf("%d",&c);
    }while(c==1);
    getch();
}
void insertq()
{
```

```

int v;
if(rear==size-1)
printf("queue overflow\n");
else
{
if(front==-1)
front=0;
rear++;
printf("enter the element into the queue\n");
scanf("%d",&v);
queue[rear]=v;
}
}
void deleteq()
{
if(front==-1)
printf("queue underflow\n");
else
{
if(front==rear)
{
printf("the deleted element from the queue is %d",queue[front]);
front=-1;
rear=-1;
}
else
{
printf("the deleted element from the queue is %d",queue[front]);
front++;
}
}
}
}
void display()
{
int i;
printf("queue elements\n");
for(i=front;i<=rear;i++)
printf("%d\n",queue[i]);
}
}

```

### Input & Output:

Queue operations

1.insert

2.delete

3.display

Enter your choice

1

Enter the element in to the queue

```

10
Do you want to continue
1
Queue operations
1.insert
2.delete
3.display
Enter your choice
1
Enter the element in to the queue
20
Do you want to continue
1
Queue operations
1.insert
2.delete
3.display
Enter your choice
3
10
20
Do you want to continue
1
Queue operations
1.insert
2.delete
3.display
Enter your choice
2
The deleted element is 10

```

**Viva Questions:**1.what are the disadvantages in linear queue?

2.how to eliminate the disadvantages using circular queue?

**7.Problem Statement:**Write a menu driven program for single linked list ADT by using the operations create, insertion at begin, insertion at middle, insertion at end, deletion at begin, deletion at middle, deletion at end, sort the list, display the list and remove the duplicate elements from the list.

**Algorithm:**

#### **1.creating a node**

Step 1: Allocating memory for the node head by using the malloc function.

Step 2: Read the head-> value

Step 3: makes the head->next value NULL.

## **2.insert at begin**

Step 1: Allocate memory for the temp node p by using the malloc function.

Step 2: read the p-> value.

Step 3: store the address of head in p-> next.

Step 4: make p as head.

## **3.insertion at middle**

Step 1: Read the position(pos) and the value to insert at that position.

Step 2: Allocate memory for node p and insert the value (p->value).

Step 2: Traverse the list from head until reaches to the pos.

Step 3: if the position obtained then insert the node at that position.

## **4.insertion at end**

Step 1: Allocate memory for the temp node p by using the malloc function.

Step 2: read the p-> value.

Step 3: Traverse the list until reaches to last node(q).

Step 4: Assign the address of p in q->next.

## **5.Deletion at begin**

Step 1: Initially head contains the starting node address

Step 2 : Assign the head reference value to head then head contains the address of 2<sup>nd</sup> node

## **6.Deletion at middle**

Step 1: read the element (elem) to delete in the list.

Step 2: Traverse the list from head and compare every node(q) in the list with elem if not assign q to p and q->next to p until it reaches to the elem

step 3: Assign q reference(q->next) to p reference(p->next).

## **7.Deletion at end**

Step 1: Initially head contains the starting node address

Step 2 : Traverse the list from head to Last but one node(p).

Step 3: Assign null to p reference.

### **8.Traverse the list**

Step 1: Assign the starting address to p.

Step 2: Traverse the list from p and print the value of each node until it reaches to the last node

### **9.Duplicate removal**

Step 1: Assign head address to p and p->next to q.

Step 2: do while p reaches to null

Step 3: do while q reaches to null

Compare p with q, if elements of p & q are same then delete q by assigning the p reference to q reference.

### **Requirements:**

H/W : Pentium IV.

S/W : Turbo C++ Compiler

Library : conio.h, stdio.h, alloc.h, process.h

### **Program:**

```
#include<stdio.h>
#include<conio.h>
#include<process.h>
#include<alloc.h>

void create();
void insertbegin();
void insertmiddle();
void insertend();
void deletebegin();
void deletemiddle();
void deleteend();
void traverse();
void print(struct node *);
```

```

void duplicate();

struct node
{
    int value;
    struct node *next;
}*head=NULL;

int n,i;

void main()
{
    int ch;
    char c;
    do
    {
        printf("single linked list operations\n");
        printf("1.create a node\n");
        printf("2.insert begin\n");
        printf("3.insertmiddle\n");
        printf("4.insert end\n");
        printf("5.delete begin\n");
        printf("6.delete middle\n");
        printf("7.delete end\n");
        printf("8.traverse\n");
        printf("9.duplicate removal\n");
        printf("10.exit\n");
        printf("enter your choice\n");
        scanf("%d",&ch);
        switch(ch)
        {
            case 1:create();
            break;
            case 2:insertbegin();
            break;
            case 3:insertmiddle();
            break;
            case 4:insertend();
            break;
            case 5:deletebegin();
            break;
            case 6:deletemiddle();
            break;
            case 7:deleteend();
            break;
            case 8:traverse();
            break;
            case 9:duplicate();

```



```

        break;
        case 10:exit(0);
        break;
        default:printf("enter correct choice");
    }
    printf("do you want to continue\n");
    fflush(stdin);
    scanf("%c",&c);
    }while(c=='y');

getch();
}

//create a node
void create()
{
    head=(struct node *)malloc(sizeof(struct node));
    printf("enter the value");
    scanf("%d",&head->value);
    head->next=NULL;
}

//inserting a node at begining
void insertbegin()
{
    struct node *p;
    p=(struct node *)malloc(sizeof(struct node));
    printf("enter the value to insert at begin\n");
    scanf("%d",&p->value);
    p->next=head;
    head=p;
}

//inserting a node at middle
void insertmiddle()
{
    struct node *p,*q;
    int pos,count=1,flag;
    p=(struct node *)malloc(sizeof(struct node));
    printf("enter the value to insert at middle\n");
    scanf("%d",&p->value);
    printf("enter the position\n");
    scanf("%d",&pos);
    q=head;
    while(q->next!=NULL)

```

```

{
count++;
if(count==pos)
break;
q=q->next;
}
p->next=q->next;
q->next=p;
}

```

```

//inserting a node at end
void insertend()
{
struct node *p,*q;

p=(struct node *)malloc(sizeof(struct node));
printf("enter the value to insert at end\n");
scanf("%d",&p->value);
q=head;
while(q->next!=NULL)
    q=q->next;
q->next=p;
p->next=NULL;
}

```

```

//deleting a node at begin
void deletebegin()
{
if(head==NULL)
printf("there is no item to delete\n");
else if(head->next==NULL)
head=NULL;
else
head=head->next;
}

```

```

//deleting a node at middle
void deletemiddle()
{
struct node *q,*p;
int ele;
if(head==NULL)
printf("there is no item to delete\n");
else

```

```

{
printf("enter the element to delete\n");
scanf("%d",&ele);
q=head;
while(q->value==ele)
{
P=q;
q=q->next;
}
p->next=q->next;
}
}

```

```

//deleting a node at end
void deleteend()
{
struct node *p,*q;
if(head==NULL)
printf("there is no item to delete\n");
else if(head->next==NULL)
head=NULL;
else
{
q=head;
while(q->next!=NULL)
{
p=q;
q=q->next;
}
p->next=NULL;
}
}

```

```

//traversing the list
void traverse()
{
struct node *p;
p=head;
while(p->next!=NULL)
{
printf("%d\n",p->value);
p=p->next;
}
printf("%d\n",p->value);

```

```
}
```

```
//duplicate removal in the list  
void duplicate()  
{  
    struct node *p,*q,*r;  
    p=head;  
    while(p!=NULL)  
    {  
        q=p->next;  
        r=p;  
        while(q!=NULL)  
        {  
            if(p->value==q->value)  
            {  
                r->next=r->next->next;  
            }  
            r=q;  
            q=q->next;  
        }  
        p=p->next;  
    }  
    traverse();  
}
```

### **Input & Output:**

Single linked list operations

1.creating a node

2.insert begin

3.insert middle

4.insert end

5.delete begin

6.delete middle

7.delete end

8.traversing

9.duplicate removal

10.exit

Enter your choice

1

Enter the value

10

Do you want to continue Y

Single linked list operations

1.creating a node

2.insert begin

3.insert middle

4.insert end

5.delete begin

6.delete middle

7.delete end

8.traversing

9.duplicate removal

10.exit

Enter your choice

2

Enter the value to insert at begin

20

Do you want to continue Y

Single linked list operations

1.creating a node

2.insert begin

3.insert middle

4.insert end

5.delete begin

6.delete middle

7.delete end

8.traversing

9.duplicate removal

10.exit

Enter your choice

4

Enter the value to insert at end

30

Do you want to continue Y

Single linked list operations

1.creating a node

2.insert begin

3.insert middle

4.insert end

5.delete begin

6.delete middle

7.delete end

8.traversing

9.duplicate removal

10.exit

Enter your choice

8

20

10

30

Do you want to continue      Y

Single linked list operations

1.creating a node

2.insert begin

3.insert middle

4.insert end

5.delete begin

6.delete middle

7.delete end

8.traversing

9.duplicate removal

10.exit

Enter your choice

6

Do you want to continue      Y

Single linked list operations

1.creating a node

2.insert begin

3.insert middle

4.insert end

- 5.delete begin
- 6.delete middle
- 7.delete end
- 8.traversing
- 9.duplicate removal
- 10.exit

Enter your choice

8

10

30

#### **Viva Questions:**

- 1) What is a ADT?
- 2) What is linear data structure?
- 3) What is the difference between linked list and arrays?
- 4) What are the advantages of linked list?
- 5) What are the disadvantages of linked lists?

**8.Problem Statement:** Write a menu driven program for circular linked list ADT by using the operations create, insertion at begin, insertion at middle, insertion at end, deletion at begin, deletion at middle, deletion at end, display the list from the list.

**Requirements:** H/W : Pentium IV.



S/W : Turbo C++ Compiler

Library : conio.h, stdio.h, alloc.h, process.h

**Program:**

```
#include<stdio.h>
#include<conio.h>
#include<process.h>
#include<alloc.h>
void create();
void insertbegin();
void insertmiddle();
void insertend();
void deletebegin();
void deletemiddle();
void deleteend();
void traverse();
void print(struct node *);
struct node
{
    int value;
    struct node *next;
}*head;
int n,i;
void main()
{
    int ch;
    char c;
    do
    {
        printf("circular linked list operations\n");
        printf("1.create a list\n");
        printf("2.insert begin\n");
        printf("3.insertmiddle\n");
        printf("4.insert end\n");
        printf("5.delete begin\n");
        printf("6.delete middle\n");
        printf("7.delete end\n");
        printf("8.traverse\n");
        printf("9.exit\n");
        printf("enter your choice\n");
        scanf("%d",&ch);
        switch(ch)
        {
            case 1:create();
            break;
```

```

        case 2:insertbegin();
        break;
        case 3:insertmiddle();
        break;
        case 4:insertend();
        break;
        case 5:deletebegin();
        break;
        case 6:deletemiddle();
        break;
        case 7:deleteend();
        break;
        case 8:traverse();
        break;
        case 9:exit(0);
        break;
        default:printf("enter correct choice");
    }
    printf("do you want to continue\n");
    fflush(stdin);
    scanf("%c",&c);
}while(c=='y');

getch();
}

void create()
{
    printf("enter the value");
    scanf("%d",&head->value);
    head->next=head;
}

void insertbegin()
{
    struct node *p,*q;
    p=(struct node *)malloc(sizeof(struct node));
    printf("enter the value to insert at begin\n");
    scanf("%d",&p->value);
    q=head;
    while(q->next!=head)
        q=q->next;
    q->next=p;
    p->next=head;
    head=p;
}

void insertmiddle()

```

```

{
struct node *p,*q;
int pos,count=1;
p=(struct node *)malloc(sizeof(struct node));
printf("enter the value to insert at middle\n");
scanf("%d",&p->value);
printf("enter the position\n");
scanf("%d",&pos);
q=head;
while(q->next!=head)
{
count++;
if(count==pos)
break;
q=q->next;
}
p->next=q->next;
q->next=p;
}

```

```

void insertend()
{
struct node *p,*q;
p=(struct node *)malloc(sizeof(struct node));
printf("enter the value to insert at end\n");
scanf("%d",&p->value);
q=head;
while(q->next!=head)
    q=q->next;
q->next=p;
p->next=head;
}

```

```

void deletebegin()
{
struct node *p;
if(head==NULL)
printf("there is no item to delete\n");
else if(head->next==head)
head=NULL;
else
{
p=head;

while(p->next!=head)
    p=p->next;
p->next=head->next;
}
}

```

```
head=head->next;
}
}
```

```
void deletemiddle()
{
    struct node *p,*q;
    int pos,count=1;
    int ele;
    if(head==NULL)
        printf("there is no item to delete\n");
    else
    {
        printf("enter the position to delete");
        scanf("%d",&pos);
        q=head;
        while(q->next!=head)
        {
            count++;
            if(count==pos)
                break;
            q=q->next;
        }
        q->next=q->next->next;
    }
}
```

```
void deleteend()
{
    struct node *p,*q;
    if(head==NULL)
        printf("there is no item to delete\n");
    else if(head->next==head)
        head=NULL;
    else
    {
        q=head;
        while(q->next!=head)
        {
            p=q;
            q=q->next;
        }
        p->next=q->next;
    }
}
```

```
void traverse()
{
```

```

struct node *p;
p=head;
while(p->next!=head)
{
print(p);
p=p->next;
}
printf("%d\n",p->value);
}

void print(struct node *p)
{
printf("%d\n",p->value);
}

```

**Input & Output:** circular linked list operations

```

1.create a list
2.insert begin
3.insertmiddle
4.insert end
5.delete begin
6.delete middle
7.delete end
8.traverse
9.exit
enter your choice
1
Enter the value
10
Do you want to continue
Y
circular linked list operations
1.create a list
2.insert begin
3.insertmiddle
4.insert end
5.delete begin
6.delete middle
7.delete end
8.traverse
9.exit
enter your choice
2
Enter the value to insert at begin
20
Do you want to continue
Y

```

```
circular linked list operations
1.create a list
2.insert begin
3.insertmiddle
4.insert end
5.delete begin
6.delete middle
7.delete end
8.traverse
9.exit
enter your choice
2
Enter the value to insert at end
30
Do you want to continue
Y
circular linked list operations
1.create a list
2.insert begin
3.insertmiddle
4.insert end
5.delete begin
6.delete middle
7.delete end
8.traverse
9.exit
enter your choice
8
20 10 30
```

**Viva Questions:**

1. What is a ADT?
- 2.What is linear data structure?
- 3.What is the difference between linked list and arrays?
- 4.What are the advantages of circular linked list?
- 5.What is the difference between single linked list and circular linked list?

**9.Problem Statement:** Write a menu driven program for double linked list ADT by using the operations create, insertion at begin, insertion at middle, insertion at end, deletion at begin, deletion at middle, deletion at end, display the list & reverse the list.

**Requirements:** H/W : Pentium IV.

S/W : Turbo C++ Compiler

Library : conio.h, stdio.h, alloc.h, process.h

**Program:**

```
#include<stdio.h>
#include<conio.h>
#include<process.h>
#include<alloc.h>
void create();
void insertbegin();
void insertmiddle();
void insertend();
void deletebegin();
void deletemiddle();
void deleteend();
void traverse();
void nthposition();
void reverselist();
void print(struct node *);
struct node
{
    int value;
    struct node *next;
    struct node *prev;
}*head=NULL,*tail=NULL;
int n,i;
void main()
{
    int ch;
    char c;
    do
    {
        printf("double linked list operations\n1.create a list\n2.insert begin\n3.insertmiddle\n4.insert
end\n5.delete begin\n6.delete middle\n7.delete end\n8.traverse\n9.nthposition\n10.reverse list\n11.exit\n");
        printf("enter your choice\n");
        scanf("%d",&ch);
        switch(ch)
        {
            case 1:create();
            break;
            case 2:insertbegin();
            break;
            case 3:insertmiddle();
```

```

        break;
        case 4:insertend();
        break;
        case 5:deletebegin();
        break;
        case 6:deletemiddle();
        break;
        case 7:deleteend();
        break;
        case 8:traverse();
        break;
        case 9:nthposition();
        break;
        case 10:reverselist();
        break;
        case 11:exit(0);
        break;
        default:printf("enter correct choice");
    }
    printf("do you want to continue\n");
    fflush(stdin);
    scanf("%c",&c);
}while(c=='y');

getch();
}

void create()
{
    head=(struct node *)malloc(sizeof(struct node));
    if(head!=NULL)
    {
        printf("enter the value");
        scanf("%d",&head->value);
        head->next=NULL;
        head->prev=NULL;
        tail=head;
    }
    else
        printf("can not create element\n");
}

void insertbegin()
{
    struct node *p;
    p=(struct node *)malloc(sizeof(struct node));

```



```

if(p!=NULL)
{
    printf("enter the value to insert at begin\n");
    scanf("%d",&p->value);
    p->next=head;
    p->prev=NULL;
    head->prev=p;
    head=p;
}
else
    printf("cannot insert\n");
}

```

```

void insertmiddle()
{
    struct node *p,*q;
    p=(struct node *)malloc(sizeof(struct node));
    int pos,count=1;
    if(p!=NULL)
    {
        printf("enter the value to insert at middle\n");
        scanf("%d",&p->value);
        printf("enter the position\n");
        scanf("%d",&pos);
        q=head;
        while(q->next!=NULL)
        {
            count++;
            if(count==pos)
                break;
            q=q->next;
        }
        p->next=q->next;
        p->next->prev=p;
        q->next=p;
        p->prev=q;
    }
    else
        printf("can not insert\n");
}

```

```

void insertend()
{
    struct node *p;
    p=(struct node *)malloc(sizeof(struct node));

```

```

if(p!=NULL)
{
printf("enter the value to insert at end\n");
scanf("%d",&p->value);
tail->next=p;
p->prev=tail;
p->next=NULL;
tail=p;
}
else
printf("can not insert\n");
}

```

```

void deletebegin()
{
if(head==NULL)
printf("there is no item to delete\n");
else if(head->next==NULL)
{
head=NULL;
tail=NULL;
}
else
{
head=head->next;
head->prev=NULL;
}
}

```

```

void deletemiddle()
{
struct node *q;
int pos,count=1;
if(head==NULL)
printf("there is no item to delete\n");
else
{
printf("enter the element to delete\n");
scanf("%d",&pos);
q=head;
while(q->next!=tail)
{
count++;
if(count==pos)
break;
q=q->next;
}
}
}

```

```

q->next=q->next->next;
q->next->prev=q;
}
}

```

```

void deleteend()
{
if(head==NULL)
printf("there is no item to delete\n");
else if(head->next==NULL)
{
head=NULL;
tail=NULL;
}
else
{
tail=tail->prev;
tail->next=NULL;
}
}

```

```

void traverse()
{
struct node *p;
p=head;
while(p->next!=NULL)
{
printf("%d\n",p->value);
p=p->next;
}
printf("%d\n",p->value);
}

```

```

void nthposition()
{
struct node *p;
int count=1,pos;
printf("enter the position from the last\n");
scanf("%d",&pos);
p=tail;
while(p->prev!=NULL)
{
if(count==pos)
printf("the element at the %d position from the end is %d",pos,p->value);
count++;
p=p->prev;
}
}

```

```

}

void reverselist()
{
struct node *p;
p=tail;
while(p->prev!=NULL)
{
printf("%d\n",p->value);
p=p->next;
}
printf("%d\n",p->value);
}

```

### Input & Output:

double linked list operations

- 1.create a list
- 2.insert begin
- 3.insertmiddle
- 4.insert end
- 5.delete begin
- 6.delete middle
- 7.delete end
- 8.traverse
- 9.nthposition
- 10.reverse list
- 11.exit

enter your choice

1

Enter the value

10

Do you want to continue

Y

double linked list operations

- 1.create a list
- 2.insert begin
- 3.insertmiddle
- 4.insert end
- 5.delete begin
- 6.delete middle
- 7.delete end
- 8.traverse
- 9.nthposition
- 10.reverse list
- 11.exit

enter your choice

2

Enter the value to insert at begin

```
20
Do you want to continue
Y
double linked list operations
1.create a list
2.insert begin
3.insertmiddle
4.insert end
5.delete begin
6.delete middle
7.delete end
8.traverse
9.nthposition
10.reverse list
11.exit
enter your choice
2
Enter the value to insert at end
30
Do you want to continue
Y
double linked list operations
1.create a list
2.insert begin
3.insertmiddle
4.insert end
5.delete begin
6.delete middle
7.delete end
8.traverse
9.nthposition
10.reverse list
11.exit
enter your choice
8
20 10 30
```

**Viva Questions:**

1. What is a ADT?
- 2.What is linear data structure?
- 3.What is the difference between linked list and arrays?
- 4.What are the advantages of double linked list?
- 5.What is the difference between single linked list and double linked list?

**10. Problem Statement:** write a program that reads two lists of elements,prints them, sort the lists, print the sorted lists, merges the lists, print merge lists.

**Algorithm:**

**Requirements:** H/W : Pentium IV.

S/W : Turbo C++ Compiler

Library : conio.h, stdio.h, alloc.h, process.h

**Program:**

```
#include<stdio.h>
#include<conio.h>
#include<process.h>
#include<alloc.h>
struct node *list();
struct node *merge(struct node *,struct node *);
void create();
void insertbegin();
void insertmiddle();
void insertend();
void deletebegin();
void deletemiddle();
void deleteend();
void traverse(struct node *);
void print(struct node *);
void sorting();
struct node *createnode(int);
struct node
{
    int value;
    struct node *next;
}*head;
int n,i;
void main()
{
    struct node *A,*B,*C;
    printf("enter the first list");
    A=list();
    head=NULL;
    printf("enter the second list");
    B=list();
    C=merge(A,B);
    printf("first list\n");
    traverse(A);
    printf("second list\n");
    traverse(B);
    printf("merge list\n");
    traverse(C);
    getch();
}
```

```

struct node *list()
{
    int ch;
    char c;
    do
    {
        printf("single linked list operations\n");
        printf("1.create a list\n");
        printf("2.insert begin\n");
        printf("3.insertmiddle\n");
        printf("4.insert end\n");
        printf("5.delete begin\n");
        printf("6.delete middle\n");
        printf("7.delete end\n");
        printf("8.traverse\n");
        printf("9.sorting\n");
        printf("10.exit\n");
        printf("enter your choice\n");
        scanf("%d",&ch);
        switch(ch)
        {
            case 1:create();
            break;
            case 2:insertbegin();
            break;
            case 3:insertmiddle();
            break;
            case 4:insertend();
            break;
            case 5:deletebegin();
            break;
            case 6:deletemiddle();
            break;
            case 7:deleteend();
            break;
            case 8:traverse(head);
            break;
            case 9:sorting();
            break;
            case 10:exit(0);
            break;
            default:printf("enter correct choice");
        }
        printf("do you want to continue\n");
        fflush(stdin);
        scanf("%c",&c);
    }while(c=='y');
}

```

```
return(head);  
}
```

```
void create()  
{  
head=(struct node *)malloc(sizeof(struct node));  
if(head==NULL)  
printf("no space in memory\n");  
else  
{  
printf("enter the value");  
scanf("%d",&head->value);  
head->next=NULL;  
}  
}
```

```
void insertbegin()  
  
{  
struct node *p;  
p=(struct node *)malloc(sizeof(struct node));  
if(p!=NULL)  
{  
printf("enter the value to insert at begin\n");  
scanf("%d",&p->value);  
p->next=head;  
head=p;  
}  
else  
printf("cannot insert\n");  
}
```

```
void insertmiddle()  
{  
struct node *p,*q;  
int pos,count=1;  
p=(struct node *)malloc(sizeof(struct node));  
if(p!=NULL)  
{  
printf("enter the value to insert at middle\n");  
scanf("%d",&p->value);  
printf("enter the position\n");  
scanf("%d",&pos);  
q=head;  
while(q->next!=NULL)
```



```

{
count++;
if(count==pos)
break;
q=q->next;
}
p->next=q->next;
q->next=p;
}
else
printf("can not insert\n");
}

```

```

void insertend()
{
struct node *p,*q;
p=(struct node *)malloc(sizeof(struct node));
if(p!=NULL)
{
printf("enter the value to insert at end\n");
scanf("%d",&p->value);
q=head;
while(q->next!=NULL)
q=q->next;
q->next=p;
p->next=NULL;
}
else
printf("can not insert\n");
}

```

```

void deletebegin()
{
if(head==NULL)
printf("there is no item to delete\n");
else if(head->next==NULL)
head=NULL;
else
head=head->next;
}

```

```

void deletemiddle()
{
struct node *q;

```

```

int ele;
if(head==NULL)
printf("there is no item to delete\n");
else
{
printf("enter the element to delete\n");
scanf("%d",&ele);
q=head;
while(q->next!=NULL)
{
if(q->value==ele)
{
q->next=q->next->next;
break;
}
q=q->next;
}
if(q->next==NULL)
printf("there is no element to delete\n");
}
}

```

```

void deleteend()
{
struct node *p,*q;
if(head==NULL)
printf("there is no item to delete\n");
else if(head->next==NULL)
head=NULL;
else
{
q=head;
while(q->next!=NULL)
{
p=q;
q=q->next;
}
p->next=NULL;
}
}

```

```

void traverse(struct node *head)
{
struct node *p;
p=head;

```

```

while(p->next!=NULL)
{
print(p);
p=p->next;
}
printf("%d\n",p->value);
}

```

```

void print(struct node *p)
{
printf("%d\n",p->value);
}

```

```

void sorting()

```

```

{
struct node *p,*q;
int temp;
for(p=head;p!=NULL;p=p->next)
{
for(q=p;q!=NULL;q=q->next)
{
if(p->value>q->value)
{
temp=p->value;
p->value=q->value;
q->value=temp;
}
}
}
}

```

```

struct node *merge(struct node *A,struct node *B)
{
struct node *C=NULL,*p,*q,*E,*X;
p=A;
q=B;
while(p&&q)
{
if(p->value<q->value)
{
E=createnode(p->value);
p=p->next;
}
else if(p->value>q->value)
{
E=createnode(q->value);

```

```

q=q->next;
}
else
{
E=createnode(p->value);
p=p->next;
q=q->next;
}
if(C==NULL)
{
C=E;
X=E;
}
else
{
X->next=E;
X=E;
}
}
while(p!=NULL)
{
E=createnode(p->value);
if(C==NULL)
{
C=E;
X=E;
}
else
{
X->next=E;
X=E;
}
p=p->next;
}

while(q!=NULL)
{
E=createnode(q->value);
if(C==NULL)
{
C=E;
X=E;
}
else
{
X->next=E;
X=E;
}
}

```

```

q=q->next;
}
return(C);
}

struct node *createnode(int v)
{
struct node *E;
E=(struct node*)malloc(sizeof(struct node *));
E->value=v;
E->next=NULL;
return(E);
}

```

### Input & Output:

### Viva Questions:

- 1) What is a ADT?
- 2) What is linear data structure?
- 3) What is the difference between linked list and arrays?
- 4) What are the advantages of linked list?
- 5) What are the disadvantages of linked lists?

**11.Problem Statement:** Implement a polynomial ADT and write a program to read two polynomials and print them ,adds the polynomials , prints the sum, multiply the polynomials and print the product.(application of single linked list)

### Algorithm:

**Requirements:** H/W : Pentium IV.  
S/W : Turbo C++ Compiler  
Library : conio.h, stdio.h, alloc.h

**Program:** #include<stdio.h>  
#include<conio.h>  
#include<alloc.h>  
struct poly \*createpoly(int[],int);  
void display(struct poly \*);  
struct poly \*addpoly(struct poly \*,struct poly \*);  
struct poly \*mulpoly(struct poly \*,struct poly \*);  
struct poly  
{  
int con,pw;  
struct poly \*next;  
};  
void main()

```

{
struct poly *A,*B,*C,*D;
int a[10],b[10],i,n1,n2;
clrscr();
printf("enter the power of first polynomial\n");
scanf("%d",&n1);
printf("enter the constant values\n");
for(i=0;i<=n1;i++)
scanf("%d",&a[i]);
printf("enter the power of second polynomial\n");
scanf("%d",&n2);
printf("enter the constant values\n");
for(i=0;i<=n2;i++)
scanf("%d",&b[i]);
A=createpoly(a,n1);
display(A);
B=createpoly(b,n2);
display(B);
C=addpoly(A,B);
display(C);
D=mulpoly(A,B);
display(D);
getch();
}

```

```

struct poly *createnode(int a,int i)
{
struct poly *E;
E=(struct poly *)malloc(sizeof(struct poly));
E->con=a;
E->pw=i;
E->next=NULL;
return(E);
}
struct poly *createpoly(int c[],int n)
{
struct poly *A,*B,*C,*H=NULL;
int i;
for(i=n;i>=0;i--)
{
if(c[i]!=0)
{
A=createnode(c[i],i);
if(H==0)
{
H=A;
B=A;
}
}
}
}

```

```

else
{
B->next=A;
B=A;
}
}
}
return(H);
}

```

```

void display(struct poly *head)
{
struct poly *p;
p=head;
while(p->next!=NULL)
{
printf("%dX^%d+",p->con,p->pw);
p=p->next;
}
printf("%d",p->con);
printf("\n");
}

```

```

struct poly *addpoly(struct poly *h1,struct poly *h2)
{
struct poly *p,*q,*A,*H=NULL,*B;
p=h1;
q=h2;
while(p&&q)
{
if(p->pw>q->pw)
{
A=createnode(p->con,p->pw);
p=p->next;
}
else if(p->pw<q->pw)
{
A=createnode(q->con,q->pw);
q=q->next;
}
else
{
A=createnode(q->con+p->con,q->pw);
p=p->next;
q=q->next;
}
}
if(H==0)
{

```

```

H=A;
B=A;
}
else
{
B->next=A;
B=A;
}
}

```

```

while(p!=NULL)
{
A=createnode(p->con,p->pw);
if(H==0)
{
H=A;
B=A;
}
else
{
B->next=A;
B=A;
}
p=p->next;
}

```

```

while(q!=NULL)
{
A=createnode(q->con,q->pw);
if(H==0)
{
H=A;
B=A;
}
else
{
B->next=A;
B=A;
}
p=p->next;
}
return(H);
}

```

```

struct poly *mulpoly(struct poly *A,struct poly *B)
{

```



```

struct poly *H=NULL,*p,*q,*E,*C,*F,*D;
int flag=0;
q=A;
while(q)
{
    if(H==0)
    {
        p=B;
        while(p)
        {
            E=createnode(q->con*p->con,q->pw+p->pw);
            if(H==0)
            {
                H=E;
                C=E;
            }
            else
            {
                C->next=E;
                C=E;
            }
            p=p->next;
        }
    }
    else
    {
        p=B;
        while(p)
        {
            E=createnode(q->con*p->con,q->pw+p->pw);
            D=H;
            while(D)
            {
                if(D->pw==E->pw)
                {
                    D->con=D->con+E->con;
                    flag=1;
                    break;
                }
                D=D->next;
            }
            D=H;
            if(flag!=1)
            {
                while(D->next!=NULL)
                    D=D->next;
                D->next=E;
            }
        }
    }
}

```

```

        flag=0;
        p=p->next;
    }
    q=q->next;
}
return(H);
}

```

#### Input & Output:

```

enter the power of first polynomial 3
Enter the constant values
4
3
2
1
enter the power of first polynomial 2
Enter the constant values
5
4
3
1st polynomial=1X^3+3X^2+3X^1+4
2nd polynomial=3X^2+4X^1+5
Addition=1X^3+5X^2+7X^1+9
Multiplication=3X^5+10X^4+22X^3+34X^2+31X^1+20

```

#### Viva Questions:

1.write the applications of single linked list?

**12 a)Problem Statement:** Write a program to implement Radix Sort algorithm using Queues.

#### Requirements:

H/W : Pentium IV.

S/W : Turbo C++ Compiler

Library : conio.h, stdio.h

#### Program:

```

#include<stdio.h>
#include<conio.h>
void insertq();
void deleteq();
int a[10],n,m,i,j,rear=0,queue[10][5],z,l,d,g,h,front=0;

```

```

void main()
{
    int x,y;
    for(x=0;x<10;x++)
    for(y=0;y<5;y++)
    queue[x][y]=0;
    printf("enter the size of array and max digit number of an array\n");
    scanf("%d%d",&n,&m);
    printf("enter the array to sort");
    for(i=0;i<n;i++)
    scanf("%d",&a[i]);
    for(d=1;d<=m;d++)
    {
        //rear=0;
        for(l=0;l!=n;l++)
        {
            g=a[l];
            for(h=1;h<=d;h++)
            {
                z=g%10;
                g=g/10;
            }
            insertq();
            rear=0;
        }
        i=0;
        for(j=0;j<=9;j++)
        {
            front=0;
            while(queue[j][front]!=0)
            {
                a[i]=queue[j][front];
                deleteq();
                i++;
            }
        }
        for(i=0;i<n;i++)
        printf("%d\t",a[i]);
        printf("\n");
    }
    getch();
}

void insertq()
{
    while(queue[z][rear]!=0)
        rear++;
    queue[z][rear]=a[l];
}

```

```

}

void deleteq()
{
queue[j][front]=0;
front++;
}

```

**Input & Output:** enter the size of an array

```

5
Enter array elements
2
4
3
5
1
The sorted list
2
2
3
4
5

```

**Viva Questions:**1.how to sort the elements using radix sort?

**12. b).Problem Statement:** Write a program to implement Shell Sort algorithm.

**Requirements:**

|         |   |                    |
|---------|---|--------------------|
| H/W     | : | Pentium IV.        |
| S/W     | : | Turbo C++ Compiler |
| Library | : | conio.h, stdio.h   |

**Algorithm:**

Step 1:We should start sorting by comparing and ordering the **one's** digits.

Step 2:Notice that the numbers were added onto the list in the order that they were found, which is why the numbers appear to be unsorted in each of the sublists above. Now, we gather the sublists

Step 3: Now, the sublists are created again, this time based on the **ten's** digit.

Step 4: Finally, the sublists are created according to the **hundred's** digit:

**Program:**

```
#include<stdio.h>
#include<conio.h>
void shallsort();
int n,a[10];
void main()
{
    int i;
    printf("enter the size of array\n");
    scanf("%d",&n);
    printf("enter the array\n");
    for(i=0;i<n;i++)
        scanf("%d",&a[i]);
    shallsort();
    printf("sorted list\n");
    for(i=0;i<n;i++)
        printf("%d\t",a[i]);
    getch();
}
void shallsort()
{
    int temp,i,j,d;
    for(d=n/2;d>=1;d=d/2)
    {
        for(i=d;i<n;i++)
        {
            temp=a[i];
            for(j=i;j>=d;j=j-d)
            {
                if(temp<a[j-d])
                    a[j]=a[j-d];
                else
                    break;
            }
            a[j]=temp;
        }
    }
}
```

**Input & Output:**

```
enter the size of an array
5
Enter array elements
2
```

4

3

5

1

The sorted list

2

2

3

4

5

**Viva Questions:** 1. explain the procedure for shell sort?

**12 c). Problem Statement:** Write a program to implement heap Sort algorithm.

**Requirements:**

H/W : Pentium IV.

S/W : Turbo C++ Compiler

Library : conio.h, stdio.h

**Algorithm:** procedure *downheap* ( $v$ )

Input: semi-heap with root  $v$

Output: heap (by rearranging the vertex labels)

Method: while  $v$  does not have the heap property do  
    1. choose direct descendant  $w$  with maximum label  $a(w)$   
    exchange  $a(v)$  and  $a(w)$   
    set  $v := w$

procedure *buildheap*

Input: almost complete binary tree  $T$  of depth  $d(T)$  with vertex labelling  $a$

Output: heap (by rearranging the vertex labels)

Method: for  $i := d(T) - 1$  downto 0 do  
    1. for all inner vertices  $v$  of depth  $d(v) = i$  do

### 1. *downheap* (*v*)

A call of *buildheap* is the first step of procedure *heapsort*, which can now be written down as follows:

procedure *heapsort*

Input: almost complete binary tree with root *r* and vertex labelling *a*

Output: vertex labels in descending order

Method: *buildheap*

while *r* is not a leaf do

1. output *a(r)*

choose leaf *b* of maximum depth

write label *a(b)* to *r*

delete leaf *b*

*downheap* (*r*)

output *a(r)*

### Program:

```
#include<stdio.h>
```

```
#include<conio.h>
```

```
#include<alloc.h>
```

```
void heap(int *,int);
```

```
void main()
```

```
{
```

```
int *x,i,n;
```

```
int temp;
```

```
clrscr();
```

```

printf("Heap Sort");

printf("Enter How many Numbers : ");

scanf("%d",&n);

x = (int *)malloc(n * sizeof(int));

for(i=0;i<n;i++)

{

    scanf("%d",&x[i]);

}

heap(x,n);

for(i=n-1;i>=1;i--)

{

    temp = x[i];

    x[i] = x[0];

    x[0] = temp;

    heap(x,i-1);

}

printf("Resultant Array");

for(i=0;i<n;i++)

    printf("%d    ",x[i]);

free(x);

getch();

}

void heap(int *a,int n)

{

```



```

int i,temp;
for(i=n/2;i>=0;i--)
{
    if(a[(2*i)+1] < a[(2*i)+2] && (2*i+1)<=n && (2*i+2)<=n)
    {
        temp = a[(2*i)+1];
        a[(2*i)+1] = a[(2*i)+2];
        a[(2*i)+2] = temp;
    }
    if(a[(2*i)+1] > a[i] && (2*i+1)<=n && i<=n)
    {
        temp = a[(2*i)+1];
        a[(2*i)+1] = a[i];
        a[i] = temp;
    }
}

```

**Input & Output:** enter the size of an array

5

Enter array elements

2

4

3

5

1

The sorted list

2

2

3

4

5

**Viva Questions:**1.explain the procedure for heap sort?

**12 d).Problem Statement:** Write a program to implement MERGE SORT algorithm.

**Requirements:**

H/W : Pentium IV.

S/W : Turbo C++ Compiler

Library : conio.h, stdio.h

**Algorithm:**

**1. Divide Step**

If given array  $A$  has zero or one element, return  $S$ ; it is already sorted. Otherwise, divide  $A$  into two arrays,  $A_1$  and  $A_2$ , each containing about half of the elements of  $A$ .

**2. Recursion Step**

Recursively sort array  $A_1$  and  $A_2$ .

**3. Conquer Step**

Combine the elements back in  $A$  by merging the sorted arrays  $A_1$  and  $A_2$  into a sorted sequence.

**Program:**

```
#include<stdio.h>

#include<conio.h>

void msort(int [],int [],int ,int );

void merge(int [],int [],int ,int ,int );

int n,a[10],b[10];

void main()

{

int i;

printf("enter the size of array\n");

scanf("%d",&n);

printf("enter the array\n");

for(i=0;i<n;i++)

scanf("%d",&a[i]);
```

```

msort(a,b,0,n-1);

printf("sorted list\n");

for(i=0;i<n;i++)

printf("%d\t",a[i]);

getch();

}

void msort(int a[],int b[],int begin,int end)

{

int center;

if(begin<end)

{

center=(begin+end)/2;

msort(a,b,begin,center);

msort(a,b,center+1,end);

merge(a,b,begin,center+1,end);

}

}

void merge(int a[],int b[],int lpos,int rpos,int rightend)

{

int i,leftend,num,tpos;

leftend=rpos-1;

tpos=lpos;

num=rightend-lpos+1;

while(lpos<=leftend&& rpos<=rightend)

{

```

```
    if(a[lpos]<=a[rpos])
    {
        b[tpos]=a[lpos];
        tpos++;
        lpos++;
    }
    else
    {
        b[tpos]=a[rpos];
        tpos++;
        rpos++;
    }
}
while(lpos<=leftend)
{
    b[tpos]=a[lpos];
    tpos++;
    lpos++;
}
while(rpos<=rightend)
{
    b[tpos]=a[rpos];
    tpos++;
    rpos++;
}
```

```
for(i=0;i<num;i++,rightend--)  
a[rightend]=b[rightend];  
}
```

**Input & Output:** enter the size of an array

```
5  
Enter array elements  
2  
4  
8  
5  
1  
The sorted list  
2  
2  
4  
5  
8
```

#### **Viva Questions:**

1. which ADT we used in merge sort?
2. what is sorting?
3. What are the Applications of sorting?

**12 e). Problem Statement:** write a program to implement quick sort algorithm.

#### **Requirements:**

H/W : Pentium IV.  
S/W : Turbo C++ Compiler  
Library : conio.h, stdio.h

#### **Algorithm:**

Step 1: Choose a key element in the list which is called a pivot.

Step 2: Reorder the list with the rule that all elements which are less than the pivot come before the pivot and so that all elements greater than the pivot come after it. After the partitioning, the pivot is in its final position.

Step 3: Recursively reorder two sub-lists: the sub-list of lesser elements and the sub-list of greater elements.

**Program:**

```
#include<stdio.h>

#include<conio.h>

void quicksort(int[],int,int);

void exchange(int[],int,int);

int n,a[10];

void main()

{

int i;

printf("enter the size of array\n");

scanf("%d",&n);

printf("enter the array\n");

for(i=0;i<n;i++)

scanf("%d",&a[i]);

quicksort(a,0,n-1);

printf("sorted list\n");

for(i=0;i<n;i++)

printf("%d\t",a[i]);

getch();

}

void quicksort(int a[],int begin,int end)
```

```

{
int i,j,pivot,c;
if(begin<end)
{
i=begin;
j=end;
c=(i+j)/2;
exchange(a,c,i);
pivot=begin;
while(i<j)
{
while(a[i]<=a[pivot])
i++;
while(a[j]>a[pivot])
j--;
if(i<j)
exchange(a,i,j);
}
exchange(a,pivot,j);
quicksort(a,begin,j-1);
quicksort(a,j+1,end);
}
}

void exchange(int a[],int i,int j)
{

```

```

int temp;

temp=a[i];

a[i]=a[j];

a[j]=temp;

}

```

**Input & Output:** enter the size of an array

```

5
Enter array elements
2
4
8
5
1
The sorted list
2
2
4
5
8

```

### **Viva Questions:**

1. which ADT we used in quick sort?
2. what is sorting?
3. What are the Applications of sorting?

### **14. C program for open addressing**

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define TABLE_SIZE 10

// Structure for a key-value pair
struct KeyValuePair {
    char key[50];
    char value[50];
};

```



```

// Structure for the hash table
struct HashTable {
    int size;
    int itemCount;
    struct KeyValuePair* table;
};

// Function to create a new key-value pair
struct KeyValuePair createKeyValuePair(const char* key, const char* value) {
    struct KeyValuePair newPair;
    strcpy(newPair.key, key);
    strcpy(newPair.value, value);
    return newPair;
}

// Function to create a new hash table
struct HashTable createHashTable(int size) {
    struct HashTable newTable;
    newTable.size = size;
    newTable.itemCount = 0;
    newTable.table = (struct KeyValuePair*)calloc(size, sizeof(struct KeyValuePair));
    return newTable;
}

// Hash function for strings (djb2 algorithm)
unsigned long hashFunction(const char* str) {
    unsigned long hash = 5381;
    int c;

    while ((c = *str++) != '\0') {
        hash = ((hash << 5) + hash) + c; // hash * 33 + c
    }

    return hash;
}

// Function to resolve collisions using linear probing
int linearProbe(int index, int attempt, int size) {
    return (index + attempt) % size;
}

// Function to insert a key-value pair into the hash table using linear probing
void insert(struct HashTable* hashTable, const char* key, const char* value) {
    if (hashTable->itemCount >= hashTable->size) {
        printf("Hash table is full. Cannot insert more items.\n");
    }
}

```

```

    return;
}

unsigned long index = hashFunction(key) % hashTable->size;
int attempt = 0;

// Probe until an empty slot is found
while (hashTable->table[index].key[0] != '\0') {
    attempt++;
    index = linearProbe(index, attempt, hashTable->size);
}

// Insert the new pair
hashTable->table[index] = createKeyValuePair(key, value);
hashTable->itemCount++;
}

// Function to retrieve the value associated with a key
const char* retrieve(struct HashTable* hashTable, const char* key) {
    unsigned long index = hashFunction(key) % hashTable->size;
    int attempt = 0;

    // Probe until the key is found or an empty slot is encountered
    while (strcmp(hashTable->table[index].key, key) != 0 && hashTable->table[index].key[0] != '\0') {
        attempt++;
        index = linearProbe(index, attempt, hashTable->size);
    }

    if (strcmp(hashTable->table[index].key, key) == 0) {
        return hashTable->table[index].value; // Key found, return the value
    } else {
        return "Key not found"; // Key not found
    }
}

// Function to display the contents of the hash table
void displayHashTable(struct HashTable* hashTable) {
    for (int i = 0; i < hashTable->size; i++) {
        printf("[%d] -> (%s, %s)\n", i, hashTable->table[i].key, hashTable->table[i].value);
    }
}

int main() {
    struct HashTable hashTable = createHashTable(TABLE_SIZE);

    // Insert key-value pairs

```

```

insert(&hashTable, "Red", "#ff0000");
insert(&hashTable, "Green", "#008000");
insert(&hashTable, "Blue", "#0000FF");
insert(&hashTable, "Yellow", "#FFFF00");
insert(&hashTable, "Orange", "#FFA500");

// Display the hash table
printf("Hash Table:\n");
displayHashTable(&hashTable);

// Retrieve and print values for specific keys
printf("\nValue for key 'Red': %s\n", retrieve(&hashTable, "Red"));
printf("Value for key 'Yellow': %s\n", retrieve(&hashTable, "Yellow"));
printf("Value for key 'White': %s\n", retrieve(&hashTable, "White"));

return 0;
}

```

### 15. Problem statement :Implement binary tree ADT

**Requirements:** H/W : Pentium IV.

S/W : Turbo C++ Compiler

Library : conio.h, stdio.h

#### Program:

```

#include<stdio.h>
#include<conio.h>
//#include<alloc.h>
void push(int);
int pop();
void insertion();
void deletion();
int searching(char);
void preorder(int);
void inorder(int);
void postorder(int);
char a[20];
int al[20],ar[20],i,stack[10],top=0;
void main()
{
clrscr();
for(i=1;i<20;i++)
{
a[i]=NULL;
al[i]=0;

```

```

ar[i]=0;
}
int m,item;
char c,ch1,ch;
do
{
printf("binary tree operations\n");
printf("1.searching\n");
printf("2.insertion\n");
printf("3.deletion\n");
printf("4.traversals\n");
printf("enter your choice\n");
scanf("%d",&ch);
switch(ch)
{
case 1: printf("enter the item to search");
fflush(stdin);
scanf("%c",&item);
m=searching(item);
if(m!=0)
printf("the searching element is in position %d\n",m);
else
printf("the search element is not in the tree\n");
break;
case 2:insertion();
break;
case 3:deletion();
break;
case 4:printf("a.preorder traversals\n");
printf("b.inorder traversals\n");
printf("c.postorder traversals\n");
printf("enter your choice\n");
fflush(stdin);
scanf("%c",&ch1);
switch(ch1)
{
case 'a':preorder(1);
break;
case 'b':inorder(1);
break;
case 'c':postorder(1);
break;
default:printf("enter correct choice\n");
}
break;
default:printf("enter correct choice\n");
}
printf("do you want to continue yes=y no=n\n");

```

```

fflush(stdin);
scanf("%c",&c);
}while(c=='y');
getch();
}
int searching(char item)
{
for(i=1;i<20;i++)
{
if(a[i]==item)
return(i);
}
return(0);
}

```

```

void insertion()
{
int s,elem,item;
char pos;
i=1;
if(a[i]==NULL)
{
printf("enter the element to insert\n");
fflush(stdin);
scanf("%c",&elem);
a[i]=elem;
al[i]=0;
ar[i]=0;
}
else
{
printf("enter insertion position");
fflush(stdin);
scanf("%c",&item);
s=searching(item);
if(s==0)
printf("there is no position to insert\n");
else
{
printf("enter the left or right position to insert the element");
fflush(stdin);
scanf("%c",&pos);
printf("enter the element to insert\n");
fflush(stdin);
scanf("%c",&elem);
if(pos=='l')
{
if(a[2*s]==NULL)

```

```

{
a[2*s]=elem;
al[s]=2*s;
}
else
printf("the left position already contains element,so can not insert element\n");
}
else
{
if(a[2*s+1]==NULL)
{
a[2*s+1]=elem;
ar[s]=2*s+1;
}
else
printf("the right position already contains element,so can not insert element\n");
}
}
}
}

```

```

void deletion()
{
int item,s,x;
printf("enter the item to delete\n");
fflush(stdin);
scanf("%c",&item);
s=searching(item);
if(s==0)
printf("there is no item to delete\n");
else
{
if(a[2*s]==NULL&& a[2*s+1]==NULL)
{
a[s]=NULL;
x=s/2;
if(al[x]==s)
al[x]=0;
else
ar[x]=0;
printf("deletion is successfull\n");
}
else
printf("this is not a leaf node\n");
}
}
}

```

```

void push(int k)
{
top++;
stack[top]=k;
}
int pop()
{
int z;
z=stack[top];
top--;
return(z);
}
void preorder(int root)
{
int p;
push(root);
while(top!=-1)
{
p=pop();
if(p!=NULL)
{
printf("%c",a[p]);
push(ar[p]);
push(al[p]);
}
}
}

```

```

/*void inorder(int p)
{
top=0;
stack[top]=NULL;
push(p);
while(top!=0)
{
if(p!=NULL)
{
push(ar[p]);
push(al[p]);
}
p=pop();
printf("%c",a[p]);
}
} */
void inorder(int root)
{
int p;
p=root;

```

```

top=0;
stack[top]=NULL;
in:while(p!=NULL)
{
    push(p);
    p=al[p];
}
p=pop();
while(p!=NULL)
{
    printf("%c",a[p]);
    if(ar[p]!=NULL)
    {
        p=ar[p];
        goto in;
    }
    p=pop();
}
}
void postorder(int p)
{
    top=0;
    stack[top]=NULL;
label:while(p!=NULL)
{
    push(p);
    if(ar[p]!=NULL)
        push(-ar[p]);
    p=al[p];
}
p=pop();
while(p>0)
{
    printf("%c",a[p]);
    p=pop();
}
if(p<0)
{
    p=-p;
    goto label;
}
}

```

#### **Input/output:**

Enter binary tree operations

- 1.searching
- 2.insertion
- 3.deletion
- 4.traversals



Enter your option  
2  
Enter element  
10  
Do you want continue  
Y  
Enter binary tree operations  
1.searching  
2.insertion  
3.deletion  
4.traversals  
Enter your option  
2  
Enter insertion position1  
Enter left or right to insert element l  
Enter the element to insert  
20  
Do you want continue  
Y  
Enter binary tree operations  
1.searching  
2.insertion  
3.deletion  
4.traversals  
Enter your option  
4  
10  
20

**Viva questions:**

1.what are the different traversals used in binary trees?

**16).write a program to read postfix arithmetic expression build expression tree and perform tree traversals**

**Requirements:** H/W : Pentium IV.  
S/W : Turbo C++ Compiler  
Library : conio.h, stdio.h

**Program:**

```
#include<stdio.h>
#include<conio.h>
#include<malloc.h>
#include<ctype.h>
```

```

#include<math.h>
int evaluate(int ,int ,char );
int treeevaluation(struct etree*);
void preorder(struct etree*);
void inorder(struct etree*);
void postorder(struct etree*);
struct etree *createnode(char);
void push(struct etree*);
struct etree* pop();
struct etree
{
char data;
struct etree *prev;
struct etree *next;
}*root=NULL,*stack[20];
int top=-1;
void main()
{
clrscr();
struct etree *e;
struct etree *m,*n;
char exp[20],choice;
int i,ch,value;
printf("enter the expression\n");
gets(exp);
for(i=0;exp[i]!='\0';i++)
{
//if(exp[i]>='a'&&exp[i]<='z')
if(isdigit(exp[i]))
{
e=createnode(exp[i]);
push(e);
}
else
{
e=createnode(exp[i]);
m=pop();
n=pop();
e->prev=n;
e->next=m;
push(e);
}
}
root=e;
do
{
printf("enter your choice\n");
printf("1.preorder traversals\n");

```

```

printf("2.inorder traversals\n");
printf("3.postorder traversals\n");
printf("enter your choice\n");
scanf("%d",&ch);
switch(ch)
{
case 1:preorder(root);
        break;
case 2:inorder(root);
        break;
case 3:postorder(root);
        break;
default:printf("enter correct choice\n");
}
printf("do you want to continue\n");
fflush(stdin);
scanf("%c",&choice);
}while(choice=='y');
value=treetreeevaluation(root);
printf("expression evaluation=%d",value);
getch();
}

struct etree *createnode(char exp)
{
struct etree *x;
x=(struct etree *)malloc(sizeof(struct etree));
x->data=exp;
x->prev=NULL;
x->next=NULL;
return(x);
}

void push(struct etree *a)
{
top++;
stack[top]=a;
}

struct etree* pop()
{
struct etree *p;
p=stack[top];
top--;
return(p);
}

void preorder(struct etree *q)
{
printf("%c\t",q->data);
if(q->prev) preorder(q->prev);
if(q->next) preorder(q->next);
}

```

```

}
void inorder(struct etree *q)
{
    if(q->prev) inorder(q->prev);
    printf("%c\t",q->data);
    if(q->next) inorder(q->next);
}
void postorder(struct etree *q)
{
    if(q->prev) postorder(q->prev);
    if(q->next) postorder(q->next);
    printf("%c\t",q->data);
}
int treeevaluation(struct etree *x)
{
    struct etree *y,*z;
    int d1,d2,value;
    char op;
    if(x!=NULL)
    {
        y=x->prev;
        z=x->next;
        if(isdigit(y->data))
            d1=y->data-48;
        else
            d1=treeevaluation(y);
        if(isdigit(z->data))
            d2=z->data-48;
        else
            d2=treeevaluation(z);
        op=x->data;
        value=evaluate(d1,d2,op);
        return(value);
    }
}

```

```

int evaluate(int d1,int d2,char op)
{
    int r;
    if(op=='+')
        r=d1+d2;
    else if(op=='-')
        r=d1-d2;
    else if(op=='*')
        r=d1*d2;
    else if(op=='/')
        r=d1/d2;
    else

```

```

r=pow(d1,d2);
return(r);
}

```

**input output:**

```

enter the expression
123*+
Enter your choice
1.preorder traversal
2.inorder traversal
3.postorder traversal1
+1*23
Do you want to continue
N
Expression tree evaluation=7

```

**Viva questions:**

1.which abstract data type we used in expression tree creation?

**17) problem statement: implement binary search tree ADT and write a program that interactively allows insertion,deletion ,find minimum,find maximumfind operations.**

**Requirements:** H/W : Pentium IV.  
S/W : Turbo C++ Compiler  
Library : conio.h, stdio.h

**Program:**

```

#include<stdio.h>
#include<conio.h>
#include<alloc.h>
void searching(int);
void findmin(struct bstree*);
void findmax(struct bstree*);
void insertion();
void deletion();
void preorder(struct bstree*);
void inorder(struct bstree*);
void postorder(struct bstree*);
struct bstree* createnode(int);
void create(struct bstree*);
struct bstree
{
int data;

```

```

struct bstree *prev;
struct bstree *next;
}*root=NULL;
struct bstree *p,*q,*ptr;
int a,in[10],k=0,min,max;
void main()
{
int ch,item;
char c,ch1;
do
{
printf("binary search(sort) tree operations\n");
printf("1.searching\n");
printf("2.insertion\n");
printf("3.deletion\n");
printf("4.traversals\n");
printf("5.findminimum\n");
printf("6.findmaximum\n");
printf("enter your choice\n");
scanf("%d",&ch);
switch(ch)
{
case 1: printf("enter the item to search");
scanf("%d",&item);
searching(item);
break;
case 2:insertion();
break;
case 3:deletion();
break;
case 4:printf("5.preorder traversals\n");
printf("6.inorder traversals\n");
printf("7.postorder traversals\n");
printf("enter your choice\n");
fflush(stdin);
scanf("%c",&ch1);
switch(ch1)
{
case 'a':preorder(root);
break;
case 'b':inorder(root);
break;
case 'c':postorder(root);
break;
default:printf("enter correct choice\n");
}
break;
case 5: min=root->data;

```

```

        findmin(root);
        printf("minimum value in binary search tree=%d\n",min);
        break;
case 6: max=root->data;
        findmax(root);
        printf("maximum value in binary search tree=%d\n",max);
        break;
default:printf("enter correct choice\n");
}
printf("do you want to continue yes=y no=n\n");
fflush(stdin);
scanf("%c",&c);
}while(c!='y');
getch();
}
void findmin(struct bstree *q)
{
if(q->data<min)
min=q->data;
if(q->prev) findmin(q->prev);
if(q->next) findmin(q->next);
}
void findmax(struct bstree *q)
{
if(q->data>max)
max=q->data;
if(q->prev) findmax(q->prev);
if(q->next) findmax(q->next);
}
void searching(int item)
{
p=root;
while(p)
{
if(p->data>item)
{
q=p;
p=p->prev;
}
else if(p->data<item)
{
q=p;
p=p->next;
}
else
{
ptr=p;
break;
}
}
}

```

```

}
}
printf("the search element is %d",ptr->data);
}

```

```

void insertion()
{
int n,i,x;
struct bstree *p,*e;
printf("enter the value\n");
scanf("%d",&x);
e=createnode(x);
if(root==NULL)
root=e;
else

```

```

{
p=root;
while(p)
{
if(p->data>e->data)
{
if(p->prev!=NULL)
p=p->prev;
else
{
p->prev=e;
break;
}
}
else
{
if(p->next!=NULL)
p=p->next;
else
{
p->next=e;
break;
}
}
}
}
}
}

```

```

void deletion()
{
struct bstree *ptr1,*q1;
int item,j,g;
p=root;
printf("enter the item to be deleted\n");

```



```

scanf("%d",&item);
searching(item);
if(ptr->prev==NULL&&ptr->next==NULL)
{
if(q->prev==ptr)
q->prev=NULL;
else
q->next=NULL;
}
else if((ptr->prev!=NULL&&ptr->next==NULL)|| (ptr->next!=NULL&&ptr->prev==NULL))
{
if(ptr->prev!=NULL)
q->next=ptr->prev;
else
q->next=ptr->next;
inorder(root);
}
else
{
ptr1=ptr;
q1=q;
inorder(root);
for(j=0;j<k;j++)
{
if(ptr->data==in[j])
{
j++;
g=in[j];
searching(g);
break;
}
}
if(q1->prev->data==ptr1->data)
{
q1->prev=ptr;
ptr->prev=ptr1->prev;
}
else
q1->next=ptr;
}
}

```

```

struct bstree* createnode(int a)
{
struct bstree *p;
p=(struct bstree*)malloc(sizeof(struct bstree));
p->data=a;
p->prev=NULL;

```

```

p->next=NULL;
return(p);
}

void preorder(struct bstree *q)
{
if(q)
{
printf("%d\t",q->data);
if(q->prev) preorder(q->prev);
if(q->next) preorder(q->next);
}
}
void inorder(struct bstree *q)
{
if(q->prev) inorder(q->prev);
printf("%d\t",q->data);
in[k++]=q->data;
if(q->next) inorder(q->next);
}
void postorder(struct bstree *q)
{
if(q->prev) postorder(q->prev);
if(q->next) postorder(q->next);
printf("%d\t",q->data);
}

```

### input output:

binary search (sort) operations

- 1.searching
- 2.insertion
- 3.deletion
- 4.traversals
- 5.find min
- 6.find max

Enter your choice

2

Enter value

10

Do you want to continue

Y

binary search (sort) operations

- 1.searching
- 2.insertion
- 3.deletion
- 4.traversals
- 5.find min
- 6.find max

Enter your choice

2

Enter value

20

Do you want to continue

Y

binary search (sort) operations

1.searching

2.insertion

3.deletion

4.traversals

5.find min

6.find max

Enter your choice

4

10

20

Do you want to continue

n

### **Viva questions:**

1.define binary search tree?

**18.problem statement:** Implement AVL tree ADT and Write a program that interactively allows:insertion,deletion,find minimum and find maximum

**Requirements:** H/W : Pentium IV.

S/W : Turbo C++ Compiler

Library : conio.h, stdio.h

### **Program:**

```
#include<stdio.h>
```

```
#include<conio.h>
```

```
#include<malloc.h>
```

```
struct avltree *createnode(int);
```

```
void searching(int);
```

```
int height(struct avltree *);
```

```
void insertion(struct avltree*,struct avltree*);
```

```
void deletion(struct avltree *,int);
```

```
void preorder(struct avltree *);
```

```
void inorder(struct avltree *);
```

```
void postorder(struct avltree *);
```

```
void lefttoleftrotation(struct avltree *);
```

```
void lefttorightrotation(struct avltree *);
```

```
void righttoleftrotation(struct avltree *);
```

```

void righttorightrotation(struct avltree *);
struct avltree
{
int data;
int height;
struct avltree *prev;
struct avltree *next;
}*root=NULL;
struct avltree *p,*q,*ptr;
int he,hl,hr,hd;
void main()
{
struct avltree *e;
int ch,item,h;
char ch1,c;
printf("avl tree operations\n");
do
{
printf("1.searching\n");
printf("2.height of tree\n");
printf("3.insertion\n");
printf("4.traversing\n");
printf("5.deletion\n");
printf("enter your choice\n");
scanf("%d",&ch);
switch(ch)
{
case 1: printf("enter the item to search");
scanf("%d",&item);
searching(item);
break;
case 2: h=height(root);
printf("%d",h);
break;
case 3: printf("enter the item to insert\n");
scanf("%d",&item);
e=createnode(item);
insertion(root,e);
break;
case 4:printf("a.preorder traversals\n");
printf("b.inorder traversals\n");
printf("c.postorder traversals\n");
printf("enter your choice\n");
fflush(stdin);
scanf("%c",&ch1);
switch(ch1)
{
case 'a':preorder(root);

```

```

        break;
    case 'b':inorder(root);
        break;
    case 'c':postorder(root);
        break;
    default:printf("enter correct choice\n");
}
break;
case 5:printf("enter the item to delete\n");
    scanf("%d",&item);
    deletion(root,item);
default:printf("enter correct choice\n");
}
printf("do you want to continue yes=y no=n\n");
fflush(stdin);
scanf("%c",&c);
}while(c!='y');
getch();
}

```

```

void lefttoleftrotation(struct avltree *p)
{
    struct avltree *a;
    a=p->prev;
    p->prev=a->next;
    a->next=p;
    p->height=height(p);
    a->height=height(a);
    p=a;
}

```

```

void righttorightrotation(struct avltree *p)
{
    struct avltree *a;
    a=p->next;
    p->next=a->prev;
    p->prev=p;
    a->height=height(a);
    p->height=height(p);
    p=a;
}

```

```

void lefttorightrotation(struct avltree *p)
{
    struct avltree *a;
    a=p->prev;
    righttorightrotation(a);
    lefttoleftrotation(p);
}

```

```

void righttoleftrotation(struct avltree *p)
{
    struct avltree *a;
    a=p->next;
    lefttoleftrotation(a);
    righttorightrotation(p);
}

```

```

int height(struct avltree *x)
{
    if(x==NULL)
    {
        he=0;
        return(he);
    }
    else
    {
        hl=height(x->prev);
        hr=height(x->next);
        if(hl<=hr)
            he=hr+1;
        else
            he=hl+1;
    }
    return(he);
}

```

```

void searching(int item)
{
    p=root;
    while(p)
    {
        if(p->data>item)
        {
            q=p;
            p=p->prev;
        }
        else if(p->data<item)
        {
            q=p;
            p=p->next;
        }
        else
        {
            ptr=p;
            break;
        }
    }
}

```

```

printf("the search element is %d",ptr->data);
}
void insertion(struct avltree *root,struct avltree *e)
{
int item,hr,hl,hd;
struct avltree *p,*leftp,*rightp;
if(root==NULL)
{
e->height=1;
root=e;
}
else
{
p=root;
if(e->data<p->data)
{
insertion(p->prev,e);
leftp=p->prev;
rightp=p->next;
if(rightp==NULL)
hr=0;
else
{
hr=rightp->height;
hl=leftp->height;
hd=hl-hr;
if(hd==2)
{
if(e->data<leftp->data)
lefttoleftrotation(p);
else
lefttorightrotation(p);
}
}
p->height=height(p);
}
}
}
else
{
insertion(p->next,e);
leftp=p->prev;
rightp=p->next;
if(leftp==NULL)
hl=0;
else
{
hr=rightp->height;
hl=leftp->height;
hd=hl-hr;
}
}
}
}

```

```

if(hd==-2)
{
    if(e->data>rightp->data)
        righttorightrotation(p);
    else
        righttleftrotation(p);
p->height=height(p);
}
}
}
}
}
}
}
struct avltree* createnode(int a)
{
    struct avltree *p;
    p=(struct avltree*)malloc(sizeof(struct avltree));
    p->data=a;
    p->prev=NULL;
    p->next=NULL;
    return(p);
}

```

```

void preorder(struct avltree *q)
{
    if(q)
    {
        printf("%d\t",q->data);
        if(q->prev) preorder(q->prev);
        if(q->next) preorder(q->next);
    }
}

void inorder(struct avltree *q)
{
    if(q->prev) inorder(q->prev);
    printf("%d\t",q->data);
    if(q->next) inorder(q->next);
}

void postorder(struct avltree *q)
{
    if(q->prev) postorder(q->prev);
    if(q->next) postorder(q->next);
    printf("%d\t",q->data);
}

void deletion(struct avltree *p,int item)
{
    struct avltree *ptr1,*q1;
    int item,j,g;
    searching(item);
}

```



```

if(ptr->prev==NULL&&ptr->next==NULL)
{
if(q->prev==ptr)
q->prev=NULL;
else
q->next=NULL;
}
else if((ptr->prev!=NULL&&ptr->next==NULL) || (ptr->next!=NULL&&ptr->prev==NULL))
{
if(ptr->prev!=NULL)
q->next=ptr->prev;
else
q->next=ptr->next;
inorder(root);
}
else
{
ptr1=ptr;
q1=q;
inorder(root);
for(j=0;j<k;j++)
{
if(ptr->data==in[j])
{
j++;
g=in[j];
searching(g);
break;
}
}
if(q1->prev->data==ptr1->data)
{
q1->prev=ptr;
ptr->prev=ptr1->prev;
}
else
q1->next=ptr;
}

}

```

**Input/output:**

1.searching  
2.insertion  
3.deletion  
4.traversals  
5.find min  
6.find max  
Enter your choice

2

Enter value

10

Do you want to continue

Y

AVL operations

1.searching

2.insertion

3.deletion

4.traversals

5.find min

6.find max

Enter your choice2

### Viva questions:

1.define AVL tree?

**19.** Write a C program for Graph Traversals: a) DFS b) BFS

#include<stdio.h>

```
int q[20],top=-1,front=-1,rear=-1,a[20][20],vis[20],stack[20];
int delete();
void add(int item);
void bfs(int s,int n);
void dfs(int s,int n);
void push(int item);
int pop();
```

```
void main()
{
    int n,i,s,ch,j;
    char c,dummy;
    printf("ENTER THE NUMBER VERTICES ");
    scanf("%d",&n);
    for(i=1;i<=n;i++)
    {
        for(j=1;j<=n;j++)
        {
            printf("ENTER 1 IF %d HAS A NODE WITH %d ELSE 0 ",i,j);
            scanf("%d",&a[i][j]);
        }
    }
    printf("THE ADJACENCY MATRIX IS\n");
    for(i=1;i<=n;i++)
    {
        for(j=1;j<=n;j++)
        {
            printf(" %d",a[i][j]);
        }
        printf("\n");
    }
}
```

do

{

```

for(i=1;i<=n;i++)
vis[i]=0;
printf("\nMENU");
printf("\n1.B.F.S");
printf("\n2.D.F.S");
printf("\nENTER YOUR CHOICE");
scanf("%d",&ch);
printf("ENTER THE SOURCE VERTEX :");
scanf("%d",&s);

switch(ch)
{
case 1:bfs(s,n);
break;
case 2:
dfs(s,n);
break;
}
printf("DO U WANT TO CONTINUE(Y/N) ? ");
scanf("%c",&dummy);
scanf("%c",&c);
}while((c=='y')||(c=='Y'));
}

```

//\*\*\*\*\*BFS(breadth-first search) code\*\*\*\*\*//

```

void bfs(int s,int n)
{
int p,i;
add(s);
vis[s]=1;
p=delete();
if(p!=0)
printf(" %d",p);
while(p!=0)
{
for(i=1;i<=n;i++)
if((a[p][i]!=0)&&(vis[i]==0))
{
add(i);
vis[i]=1;
}
p=delete();
if(p!=0)
printf(" %d ",p);
}
for(i=1;i<=n;i++)
if(vis[i]==0)
bfs(i,n);
}

```

```

void add(int item)
{
if(rear==19)
printf("QUEUE FULL");

```

```

else
{
if(rear== -1)
{
q[++rear]=item;
front++;
}
else
q[++rear]=item;
}
}
int delete()
{
int k;
if((front>rear)|| (front== -1))
return(0);
else
{
k=q[front++];
return(k);
}
}

```

```

//*****DFS(depth-first search) code*****//
void dfs(int s,int n)
{
int i,k;
push(s);
vis[s]=1;
k=pop();
if(k!=0)
printf(" %d ",k);
while(k!=0)
{
for(i=1;i<=n;i++)
if((a[k][i]!=0)&&(vis[i]==0))
{
push(i);
vis[i]=1;
}
k=pop();
if(k!=0)
printf(" %d ",k);
}
for(i=1;i<=n;i++)
if(vis[i]==0)
dfs(i,n);
}
void push(int item)
{
if(top==19)
printf("Stack overflow ");
else
stack[++top]=item;
}

```

```

int pop()
{
int k;
if(top==-1)
return(0);
else
{
k=stack[top--];
return(k);
}
}

```

/\* Output of BFS(breadth-first search) and DFS(depth-first search) program \*/

```

ENTER THE NUMBER VERTICES 3
ENTER 1 IF 1 HAS A NODE WITH 1 ELSE 0
ENTER 1 IF 1 HAS A NODE WITH 2 ELSE 0
ENTER 1 IF 1 HAS A NODE WITH 3 ELSE 0
ENTER 1 IF 2 HAS A NODE WITH 1 ELSE 0
ENTER 1 IF 2 HAS A NODE WITH 2 ELSE 0
ENTER 1 IF 2 HAS A NODE WITH 3 ELSE 0
ENTER 1 IF 3 HAS A NODE WITH 1 ELSE 0
ENTER 1 IF 3 HAS A NODE WITH 2 ELSE 0
ENTER 1 IF 3 HAS A NODE WITH 3 ELSE 0
THE ADJACENCY MATRIX IS
1 1 0

```

Output of BFS and DFS Program